

A PROJECT REPORT ON

**Using Gen-AI to Generate Blood Test Analysis
to Reverse Metabolic Syndrome**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Arohi Karhade	Exam No: B400050437
Isha Verma	Exam No: B400050628
Varad Gadaokar	Exam No: B400050390



**DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043**

**SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025**



CERTIFICATE

This is to certify that the project report entitled

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

Submitted by

Arohi Karhade	Exam No: B400050437
Isha Verma	Exam No: B400050628
Varad Gadaokar	Exam No: B400050390

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. J. S. Mahajan** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. J. S. Mahajan
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune

Date: 25/04/2025

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ‘Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome’.

*We would like to take this opportunity to thank **Prof. J. S. Mahajan** giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for his indispensable support, suggestions.*

*In the end our special thanks to **Mr. Prashant Karhade**, Avantza Technologies Pvt. Ltd., and all faculty members for their whole hearted cooperation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.*

Arohi Karhade
Isha Verma
Varad Gadaokar
(B.E. Computer Engg.)

ABSTRACT

Metabolic Syndrome is a set of conditions that increase the risk of heart disease, stroke, and type 2 diabetes. The conditions which lead to metabolic syndrome might include high blood sugar, high blood pressure, excess body fat, abnormal ratio of triglycerides to HDL(High Density Lipoprotein), abnormal cell width distribution, high glycated hemoglobin, insulin resistance, etc. These parameters are essential indicators of one's overall health. However, it is difficult for the general population to interpret these metrics and values in a blood test report. The aim of this research paper is to simplify medical terminologies related to metabolic syndromes, identifying key blood test parameters and generating personalized dietary and lifestyle recommendations. In this work, we aim to utilize Generative Artificial Intelligence to automate the analysis of blood test reports. To implement our system we use the Retrieval Augmented Generation method on the Llama3 base model. The RAG architecture enables the system to retrieve relevant information from a custom knowledge base consisting of books, YouTube videos and research papers. Further, the system will advise patients to seek medical attention if necessary. This approach empowers patients with actionable insights to improve their health and well-being based on real-time blood test data.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	3
1.4	Project Scope & Limitations	5
1.5	Methodologies of Problem solving	7
1.5.1	A. Data Collection and Preprocessing	7
1.5.2	B. Model Selection	7
1.5.3	C. Retrieval-Augmented Generation (RAG)	7
2	Literature Survey	9
2.1	Literature Review	10
3	Software Requirements Specification	12
3.1	Assumptions and Dependencies	13
3.1.1	Assumptions	13
3.1.2	Dependencies	13
3.2	Functional Requirements	14
3.2.1	User Authentication and Profile Management	14
3.2.2	Blood Test Report Input	14
3.2.3	Blood Test Report Analysis	14
3.2.4	Personalized Recommendations	14
3.2.5	Chatbot Interface	14
3.2.6	LLM Model Integration	14
3.2.7	Data Storage and Retrieval	15
3.2.8	Progress Tracking	15
3.3	External Interface Requirements	16
3.3.1	Web-Based Interfaces	16
3.3.2	Chat Interfaces	16
3.4	Nonfunctional Requirements	18
3.4.1	Performance Requirements	18

3.4.2	Safety and Security Requirements	18
3.4.3	Scalability	18
3.5	System Requirements	20
3.5.1	Software Requirements	20
3.5.2	Hardware Requirements	21
3.6	Analysis Models: SDLC Model to be applied	21
4	System Design	23
4.1	System Architecture	24
4.1.1	Presentation Layer	24
4.1.2	Application Layer	25
4.1.3	Business Logic Layer	25
4.1.4	Data Access Layer	25
4.1.5	Data Layer	26
4.1.6	AI Layer	26
4.2	Data Flow Diagrams	27
4.3	Entity Relationship Diagram	28
4.4	Use Case Diagram	29
4.5	Sequence Diagram	30
5	Project Plan	31
5.1	Project Estimate	32
5.1.1	Reconciled Estimates	32
5.1.2	Project Resources	32
5.2	Risk Management	34
5.2.1	Risk Identification	34
5.2.2	Risk Analysis	35
5.2.3	Overview of Risk Mitigation, Monitoring, and Man- agement	36
5.3	Project Schedule	37
5.3.1	Project Task Set	37
5.3.2	Task Network	39
5.4	Team Organization	41
5.4.1	Team Members and Roles	41
5.4.2	Management, Reporting and Communication	42
6	Project Implementation	43
6.1	Overview of Project Modules	44
6.1.1	Data Collection and Preprocessing	44
6.1.2	Model Selection	44
6.1.3	Retrieval-Augmented Generation	45

6.1.4	System Integration and Implementation	47
6.2	Tools and Technologies Used	50
6.2.1	Frontend Development	50
6.2.2	Backend Development	50
6.2.3	Artificial Intelligence and Model Integration	50
6.2.4	Database	51
6.2.5	Data Processing and Natural Language Processing	51
6.2.6	Version Control	52
6.3	Algorithms Used	53
6.3.1	Embedding Algorithm: FastEmbedEmbeddings	53
6.3.2	Vector Similarity Search	53
6.3.3	Recursive Character Text Splitter	53
6.3.4	LLM Inference (ChatOllama using LLaMA3)	53
7	Software Testing	54
7.1	Type of Testing	55
7.1.1	Unit Testing	55
7.1.2	Integration Testing	55
7.1.3	System Testing	55
7.1.4	Performance Testing	55
7.1.5	User Acceptance Testing (UAT)	55
7.2	Test Cases and Results	56
8	Results	59
8.1	Outcomes	60
8.1.1	Report Upload and Parsing	60
8.1.2	Parameter Evaluation and Categorization	60
8.1.3	LLM Response Generation	60
8.1.4	End-to-End System Integration	61
8.1.5	User Interface and Experience	61
8.1.6	Stakeholder Validation and Feedback	61
8.2	Screenshots	63
9	Conclusion	66
9.1	Conclusion	67
9.2	Future Work	68
9.3	Applications	69
10	Appendix	70
10.1	Appendix B	73
10.1.1	Paper Publication	73

Appendix D: Paper Publication	73
10.1.2 Certificates	73
Appendix E: Certificates	73
10.2 Appendix C	74
11 References	75

List of Figures

3.1	Agile Model for SDLC	22
4.1	System Architecture	24
4.2	Workflow Diagram	27
4.3	ER Diagram	28
4.4	Use Case Diagram	29
4.5	Sequence Diagram	30
6.1	RAG Pipeline	46
7.1	Result Graph	56
7.2	Results Table	58
8.1	Sign Up Page	63
8.2	Login	63
8.3	User Prompt	64
8.4	Prompt and Response	64
8.5	Prompt and Response	65
10.1	Certificates of Participation in Impetus and Concepts '25 . . .	73
10.2	Plagiarism report	74

CHAPTER 1

INTRODUCTION

1.1 Overview

Metabolic Syndrome is a group of conditions, including abdominal obesity, high blood pressure, hyperglycemia, and abnormal cholesterol levels, that increase the risk of cardiovascular diseases, stroke, and diabetes. These conditions are diagnosed through blood tests, but many individuals find it challenging to interpret the results due to complex medical terms and a lack of contextual understanding, often leading to confusion and anxiety.

This project aims to develop a Generative Artificial Intelligence (GAI) system that simplifies the interpretation of blood test reports, focusing on the detection and reversal of Metabolic Syndrome. By providing clear, personalized explanations, the system seeks to empower individuals to understand their health data and take informed actions, ultimately improving health outcomes and addressing the growing burden of Metabolic Syndrome.

To achieve this, the project implements a Retrieval-Augmented Generation (RAG) pipeline, where a Large Language Model (LLM) is integrated with a retrieval system that sources relevant information from a curated knowledge base composed of medical books, clinical research papers, and expert-reviewed healthcare resources. When a user uploads a blood test report, the system extracts key parameters, identifies abnormal values, and retrieves relevant medical context. The generative model then synthesizes this information into user-friendly explanations and provides lifestyle and dietary recommendations aimed at reversing or managing Metabolic Syndrome.

The RAG approach ensures that the responses are not only personalized and contextually relevant but also grounded in authoritative medical literature. The system is designed to support continuous learning and can be updated with new medical guidelines and research findings, enhancing its adaptability and long-term utility.

1.2 Motivation

The motivation behind this project stems from the increasing prevalence of Metabolic Syndrome, which significantly raises the risk of heart disease, stroke, and type 2 diabetes. As more individuals are affected by this condition, the need to interpret complex medical metrics, such as those presented in blood test reports, becomes crucial. Many people struggle to understand these reports due to technical jargon, making it difficult to take informed action. This project aims to simplify the understanding of these metrics while highlighting the critical role that informed dietary and lifestyle changes play

in managing and potentially reversing Metabolic Syndrome, empowering individuals to take control of their health.

1.3 Problem Definition and Objectives

Problem Definition:

Metabolic syndromes are becoming increasingly prevalent, yet most patients remain oblivious to the medical terminology used in their blood test reports. Such vagueness usually leads to confusion regarding the critical parameters in their condition, how to treat it properly, and especially through natural interventions such as diet and lifestyle changes. Poor health outcomes are commonly recorded among patients due to this uncertainty. The project will, therefore, aim to design a system that simplifies complex medical language, emphasizes the key parameters related to metabolic syndrome, and offers personalized dietary and lifestyle recommendations so patients can more effectively manage their own condition.

Objectives:

The rising prevalence of Metabolic Syndrome highlights the urgent need for tools that can make health data more accessible and actionable for the average person. This project sets out to bridge the gap between complex medical diagnostics and patient understanding by building an intelligent, user-friendly system powered by Generative AI.

- **Simplify Blood Test Interpretation**

Develop a system that translates complex medical terms in blood test reports into clear, patient-friendly language, specifically focusing on parameters associated with Metabolic Syndrome.

- **Highlight Key Health Indicators**

Identify and emphasize the most critical blood test parameters that contribute to the diagnosis and monitoring of Metabolic Syndrome, such as blood glucose, cholesterol levels, and blood pressure. The system focuses on a curated set of 30 key biomarkers that are relevant to assessing metabolic health, ensuring comprehensive and targeted insights for early detection and effective management.

- **Deliver Personalized Recommendations**

Provide tailored dietary and lifestyle advice based on each individual's health profile to support natural interventions and long-term wellness focusing on dietary and lifestyle changes.

- **Empower Patient Decision-Making**

Bridge the knowledge gap between medical professionals and patients, encouraging proactive health management and increased patient engagement.

Through these objectives, the project aims to transform the way patients engage with their blood test data—making interpretation simpler, health insights clearer, and recommendations more actionable. Ultimately, it seeks to support early intervention and better self-management, contributing to improved long-term health outcomes for individuals at risk of or living with Metabolic Syndrome.

1.4 Project Scope & Limitations

Understanding the scope and limitations of this project is essential to defining its capabilities, intended use, and boundaries. While the system is designed to enhance patient understanding and support the management of Metabolic Syndrome through AI-powered interpretation and recommendations, it operates within specific parameters. The following section outlines the key areas the project covers, as well as its current constraints, to provide a balanced view of its functionality and potential impact.

Scope:

The key areas covered under the project scope include:

- **Focus on Metabolic Syndrome:** The project is specifically designed to interpret blood test reports in the context of diagnosing and managing Metabolic Syndrome. It covers related health markers such as blood glucose, lipid profile, blood pressure, insulin resistance indicators, and inflammatory markers.
- **Personalized Interpretation:** The system provides user-friendly explanations for test results, translating complex medical terms into understandable insights tailored to the individual's health profile.
- **RAG-based GenAI Model:** The project uses a Retrieval-Augmented Generation (RAG) architecture, combining a fine-tuned Large Language Model with a medical knowledge base comprising books, research papers, and expert articles to generate context-aware, trustworthy responses.
- **Dietary and Lifestyle Recommendations:** Based on the interpreted values, the system offers personalized dietary and lifestyle changes aimed at reversing or managing Metabolic Syndrome using evidence-based practices.

Limitations:

Despite the promising results, the project has some limitations:

- **Language and Regional Constraints:** Currently, the system operates in English and may not yet support region-specific dietary suggestions or non-English-speaking users.

- **Not a Substitute for Medical Advice:** The AI-generated insights are meant for educational and supportive purposes only. Diagnosis, prescription, or treatment decisions should always involve a qualified healthcare provider.
- **Limited Personalization Based on Test Data** While personalized, the recommendations are generated based on general guidelines and may not consider unique medical histories, allergies, or prescriptions unless explicitly integrated in future versions.

1.5 Methodologies of Problem solving

The methodology for this project involves collecting and preprocessing data, selecting an appropriate model, and applying Retrieval-Augmented Generation (RAG) to interpret blood test results related to Metabolic Syndrome and provide personalized health recommendations.

1.5.1 A. Data Collection and Preprocessing

Data is sourced from three components:

- **Medical Books** (in PDF format) providing detailed knowledge on Metabolic Syndrome.
- **YouTube Videos** transcribed using OpenAI's Whisper module, featuring expert insights on managing Metabolic Syndrome.
- **Parameter Document** defining critical health markers related to Metabolic Syndrome.

The extracted data was cleaned and formatted for analysis. Text from PDFs was processed using PyMuPDF, while YouTube transcripts were cleaned by removing non-essential content.

1.5.2 B. Model Selection

Various models were considered, including OpenAI's models and Meta's **Llama-3** 8-billion parameter model. The **Llama-3 model** was chosen due to its open-source accessibility, efficiency, and ability to be customized for medical data.

1.5.3 C. Retrieval-Augmented Generation (RAG)

RAG optimizes the output of the model by referencing an external knowledge base to improve response accuracy. The process involves:

- **Vectorizing the Knowledge Base:** Text data is transformed into vector embeddings stored in ChromaDB, enabling efficient similarity searches.
- **Retrieval Pipeline:** User queries are converted into vector embeddings, which are compared against stored embeddings to retrieve the most relevant information.

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

- **Generation Pipeline:** The retrieved context is combined with the user query, and the model generates a response with personalized health recommendations.

This approach ensures users receive accurate, contextually relevant, and personalized health advice based on their blood test results, with citations provided for transparency.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Review

This literature survey explores the evolving understanding and management of metabolic syndrome (MetS), a global health concern influenced by lifestyle, diet, and physical activity. It highlights recent findings on the prevalence of MetS, effective dietary interventions, the role of physical activity, and emerging technological advancements. In particular, it emphasizes the growing role of Generative AI (GenAI) in healthcare, showcasing how modern computational tools are reshaping diagnosis, treatment, and patient care. The following papers were reviewed to understand metabolic syndrome, its lifestyle factors, and the role of GenAI in healthcare:

1. **The global epidemic of the metabolic syndrome:** The World Health Organization (WHO) defines metabolic syndrome (MetS) as a pathological condition marked by abdominal obesity, insulin resistance, hypertension, and hyperlipidemia. Originally identified in the Western world, the prevalence of MetS has expanded globally with the adoption of Western dietary and lifestyle habits. Studies indicate that urban populations in developing countries may now exhibit higher rates of MetS compared to Western nations. The rapid rise in fast food consumption, which is high in calories and low in fiber, along with reduced physical activity due to mechanized transport and sedentary leisure activities, are identified as key contributors. The syndrome significantly raises the risk of developing chronic conditions such as type 2 diabetes, coronary artery disease, stroke, and other health complications.
2. **Prevalence of metabolic syndrome among adult population in India: A systematic review and meta-analysis:** A recent study in the United States reported the prevalence of metabolic syndrome (MetS) to be approximately 22.9%. Comparable population-based studies in India revealed a wide range in prevalence, between 10% and 30%. Notably, a comprehensive study conducted between 2006 and 2010 across eleven major urban Indian cities reported a significantly higher prevalence of 35%. These findings highlight that urban populations in developing nations are especially vulnerable to MetS, largely due to lifestyle and behavioral factors. The variability in prevalence across regions and ethnicities underscores the importance of localized health strategies and interventions.
3. **Dietary and lifestyle risk factors and metabolic syndrome:** Extensive research has been conducted on dietary habits and their association with metabolic syndrome (MetS), identifying the Mediterranean

Diet as particularly beneficial. Originally introduced by Ancel Keys, this diet emphasizes high consumption of fruits, vegetables, nuts, olive oil, and fish, while limiting saturated fats, red and processed meats, refined carbohydrates, and full-fat dairy. Adherence to this dietary pattern has been shown to improve both physical and mental health, enhance quality of life, and significantly reduce risk factors associated with MetS. Specifically, it helps lower LDL cholesterol, blood glucose, and triglyceride levels, while boosting HDL cholesterol, thus offering a holistic approach to preventing and managing MetS.

4. **Generative AI for transformative healthcare:** One of the key challenges in applying deep learning to healthcare, especially in medical imaging, is the limited availability of large, labeled datasets. To address this, generative techniques such as Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) have emerged as powerful tools. These models can generate synthetic medical images that closely resemble real patient data, thereby enhancing the volume and variability of training data. This synthetic augmentation helps improve the robustness, generalizability, and accuracy of diagnostic models, making them more effective even with constrained real-world datasets. These advancements are proving crucial for the development of more reliable AI-powered tools in clinical environments.
5. **Advancing the democratization of generative artificial intelligence in healthcare:** Generative AI (GenAI), especially models based on large language architectures, has ushered in a new era of intelligent healthcare applications. Unlike traditional rule-based or deterministic systems, GenAI offers general-purpose capabilities that can interact using natural language and adapt across diverse medical tasks. Its democratization, fueled by public accessibility and minimal domain-specific constraints, makes it highly suitable for widespread deployment. GenAI can be applied in areas such as medical research, diagnosis, patient interaction, and care planning. The ability to analyze patterns from large datasets and produce coherent, human-like outputs enhances its utility in complex decision-making processes, setting it apart as a transformative technology in modern healthcare.

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

3.1.1 Assumptions

- Users are expected to have basic familiarity with interacting with chat-bots and uploading documents.
- Blood test reports uploaded by users are presumed accurate and in supported formats such as PDF or JPEG.
- Users trust the platform to securely handle their health data in accordance with applicable privacy regulations.
- The research and studies supporting the Retrieval-Augmented Generation (RAG) method are considered credible and current.

3.1.2 Dependencies

- The system utilizes the Llama API for generating responses and powering the RAG model.
- Development and fine-tuning depend on libraries like Hugging Face Transformers and TensorFlow.
- A stable internet connection is required for the seamless operation of the system.

3.2 Functional Requirements

3.2.1 User Authentication and Profile Management

The system provides secure authentication using a username and password to ensure that only authorized users can access their blood test reports and recommendations. It allows users to manage their profiles, including personal details (name, age, gender), and any medical history they wish to store.

3.2.2 Blood Test Report Input

Users can upload their blood test reports in PDF format. The system parses and stores the data for further analysis.

3.2.3 Blood Test Report Analysis

The system analyzes the uploaded blood test report data and correlates the results to identify potential metabolic syndrome risks. It compares the user's data against reference values (optimal, normal, and physically possible ranges) and flags deviations or indicators of metabolic syndrome risk factors such as high glucose, insulin resistance, and lipid imbalances.

3.2.4 Personalized Recommendations

Based on the analysis, the system generates personalized dietary and lifestyle recommendations to help reverse metabolic syndrome. These recommendations are powered by a fine-tuned LLM trained on sponsor-provided research data, books, papers, and videos.

3.2.5 Chatbot Interface

The system features a chatbot that enables real-time interaction with users, offering guidance and answering queries related to blood test results. Users can ask follow-up questions and request clarifications on their reports or recommendations.

3.2.6 LLM Model Integration

The system integrates a Retrieval-Augmented Generation (RAG) model to process natural language queries and deliver intelligent, context-aware responses.

3.2.7 Data Storage and Retrieval

All blood test reports and analysis results are stored securely in a database. Users can access and retrieve their previous reports and recommendations at any time.

3.2.8 Progress Tracking

The system tracks users' health metrics over time based on the uploaded reports and provides trend analysis to monitor progress and improvements.

3.3 External Interface Requirements

3.3.1 Web-Based Interfaces

The system provides a web-based user interface accessible via standard web browsers. This interface serves as the primary interaction point for users with our RAG model. It allows users to create multiple chats, submit their blood test reports, receive analysis, and ask any follow-up questions related to specific aspects of their reports.

The interface will include:

- **User Registration/Login:** The web interface allows users to create accounts and securely log in to access their data and previous interactions with the chatbot.
- **Dashboard:** Users have access to a dashboard that displays their blood test reports, analysis, and personalized recommendations.
- **Report Upload:** The interface provides a user-friendly file upload feature for users to submit their blood test reports.
- **Recommendations Display:** Users can view personalized dietary and lifestyle recommendations based on their blood test analysis.

3.3.2 Chat Interfaces

The system integrates seamlessly with the web-based interface to support smooth interactions and decision-making through conversational AI. Key integrations include:

- **User Input:** The chatbot accepts text input from users to initiate conversations, ask questions, and upload blood test reports.
- **Response Generation:** The chatbot provides responses based on user input, including analysis results and personalized health recommendations.
- **File Upload:** Users can upload their blood test reports in common formats (e.g., PDF, JPEG) directly through the chatbot interface.
- **Natural Language Processing:** The chatbot utilizes NLP to understand user queries and deliver coherent, contextually appropriate responses.

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

- Multi-turn Conversations: The interface supports multi-turn dialogues, enabling users to ask follow-up questions and receive further clarification.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

1. **Model Accuracy:** The RAG model achieves high accuracy in interpreting medical parameters and generating reliable health insights, ensuring that predictions and recommendations are clinically sound.
2. **Low Latency:** The system provides fast and real-time analysis of blood reports. Users receive near-instantaneous results after submitting their data, with minimal processing time.
3. **Efficiency in Resource Utilization:** The RAG model is optimized to use minimal computational resources while still delivering high-quality results. This includes effective management of memory, processing power, and GPU utilization.
4. **High Throughput:** The system efficiently processes a large volume of data inputs, especially when multiple test parameters are analyzed. It handles both structured data (numerical values from the test) and unstructured data (insights generation) seamlessly.

3.4.2 Safety and Security Requirements

1. User data, including sensitive blood test information, is handled with the highest priority for security and confidentiality.
2. Robust encryption protocols are implemented to protect data during transmission and storage.
3. Secure data storage mechanisms are employed to prevent unauthorized access and data breaches.
4. The system adheres to strict privacy standards and complies with relevant data protection regulations.
5. User data remains confidential and is not shared with or exposed to any external parties.

3.4.3 Scalability

1. **User Scalability:** The system accommodates a growing number of users, scaling seamlessly as the user base expands.

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

2. Data Scalability: The database stores and manages an increasing volume of blood test reports, user interactions, and generated recommendations without performance issues.
3. Model Scalability: The RAG model is designed to incorporate new data and adapt to emerging research findings, ensuring continuous improvement of recommendations.

3.5 System Requirements

3.5.1 Software Requirements

1. **Programming Language:** Install NVIDIA Omniverse for creating and running the virtual manufacturing environments.
2. **NVIDIA Drivers:** Ensure that the system is equipped with the latest NVIDIA drivers compatible with Omniverse for optimal GPU performance.
3. **Operating System:** Linux (Ubuntu 20.04 or later) for most server-based deployments, as it provides better performance and compatibility for RL frameworks.
4. **Programming Languages:** Python has been used for backend development, including data generation, processing, and analysis.
5. **LLM Framework:** Hugging Face Transformers is utilized for integrating and managing the large language model (LLM) capabilities within the system.
6. **AI Middleware:** LangChain is used to connect the language model with the chatbot interface and system logic, enabling smooth interactions and contextual understanding.
7. **Large Language Model:** The system integrates the LLaMA 3 8 billion parameter LLM model, to generate personalized and context-aware health insights.
8. **Chatbot Framework:** A custom chatbot framework has been integrated to support user interaction and deliver real-time, tailored responses based on blood test data.
9. **Database:** MongoDB has been employed as a secure and scalable database to store user information, uploaded blood test reports, and their corresponding analyses.
10. **Web Application Framework:** React.js has been used to build a responsive and user-friendly front-end interface, allowing users to upload reports and access personalized insights with ease.

3.5.2 Hardware Requirements

1. **CPU: 4-core** - Required to smoothly run the Spring Boot backend, manage API calls, and handle multiple user requests concurrently.
2. **RAM: 16 GB** - Ensures smooth operation of backend services, RAG query processing, and in-memory handling of blood report data.
3. **Storage: 256 GB SSD** - Provides fast read/write access to system files, temporary report uploads, model outputs, and local logging.
4. **Operating System: Windows** - Supports development and testing flexibility across platforms.

3.6 Analysis Models: SDLC Model to be applied

Agile methodology is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, and customer feedback throughout the development process. It focuses on delivering small, functional pieces of the software in shorter cycles (often called sprints), which allows teams to quickly adapt to changes and continuously improve the product based on user input.

Agile Model is the most appropriate for your project for the following reasons:

1. **Flexibility:** Given the nature of Generative AI and its applications in healthcare, requirements may evolve based on user feedback and new research findings. Agile allows for iterative development and regular updates.
2. **User-Centric Development:** The project involves a chatbot that interacts with users. Agile facilitates continuous user involvement and feedback, ensuring that the final product meets user expectations.
3. **Adaptation to Change:** With the fast-paced advancements in AI technologies and healthcare requirements, an Agile approach allows the development team to adapt quickly to changes.
4. **Continuous Integration and Testing:** Implement continuous integration practices to ensure that the new code is integrated and tested frequently. Automated testing can help identify issues early and maintain the quality of the software.

5. Backlog Management: Maintain a product backlog that includes all user stories, tasks, and enhancements. Prioritize the backlog based on user feedback, project goals, and technical feasibility
6. Regular User Feedback: At the end of each sprint, demonstrate the developed features to stakeholders and collect feedback. Use this input to refine user stories and prioritize the backlog for the next sprint.

By utilizing Agile methodology, our project can become more adaptive, responsive, and user-centric, ultimately leading to a more effective and impactful Generative AI-driven system for analyzing blood test reports. The iterative nature of Agile allows for continuous improvement and ensures that the final product aligns with the needs and expectations of its users.



Figure 3.1: Agile Model for SDLC

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

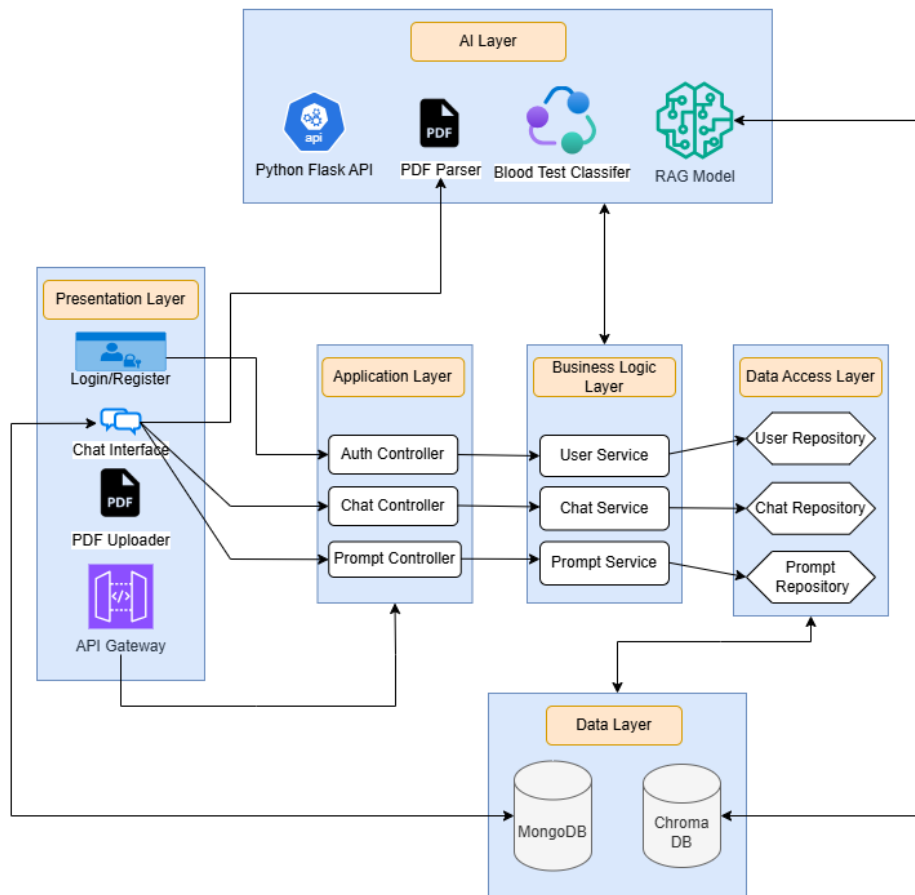


Figure 4.1: System Architecture

4.1.1 Presentation Layer

- Provides the user interface for login, registration, chat interaction, and PDF uploads.
- Acts as the entry point for users to interact with the system.
- Includes React components for user experience and frontend validation.

- Communicates with the backend through API calls and manages session flow.
- Serves as a single entry point for all frontend requests to backend services.
- Handles routing, security checks, and request filtering.
- Ensures proper communication between frontend and microservices.

4.1.2 Application Layer

- Contains REST controllers for authentication, chat, and prompt handling.
- Translates HTTP requests into service layer method calls.
- Implements request validation and response formatting.
- Coordinates frontend requests and sends appropriate data back.

4.1.3 Business Logic Layer

- Implements the core logic of user management, chat handling, and prompt processing.
- Contains UserService, ChatService, and PromptService for handling operations.
- Ensures proper communication between controllers and data repositories.
- Applies business rules before storing or retrieving data from the database.

4.1.4 Data Access Layer

- Provides repositories to interact with MongoDB collections.
- Includes UserRepository, ChatRepository, and PromptRepository.
-
- Offers query methods to read/write data using Spring Data MongoDB.
- Acts as a bridge between business logic and data storage.

4.1.5 Data Layer

- Stores persistent data required for application functionality.
- Uses MongoDB for storing users, chats, and prompt history.
- Integrates ChromaDB to store and retrieve vector embeddings for semantic search.
- Enables long-term storage and efficient querying of structured and vectorized data.

4.1.6 AI Layer

- Consists of core AI components like PDF Parser, Blood Test Classifier, and RAG Model.
- Responsible for extracting, classifying, and generating insights from health data.
- Connects with the backend via a Python Flask API for processing.
- Implements intelligent reasoning and personalized feedback generation.
- Exposes endpoints to accept health parameter input and return analysis.
- Acts as the backend gateway to the AI layer.
- Processes requests from Spring Boot and routes them to the relevant AI module.
- Handles PDF input, classification, and RAG-based response generation.
- Retrieval-Augmented Generation model that combines search with generative AI.
- Uses vector stores like ChromaDB to fetch relevant context before generating output.
- Produces personalized, human-readable health reports and advice.
- Powers the core of the system's intelligent response capability.

4.2 Data Flow Diagrams

A description of each major software function, along with data flow is presented.

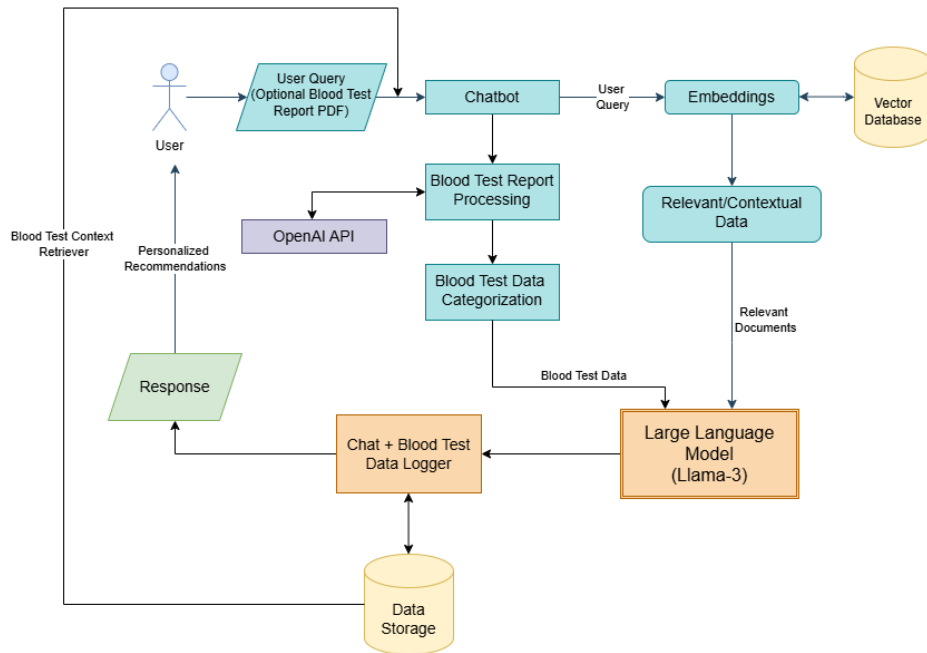


Figure 4.2: Workflow Diagram

4.3 Entity Relationship Diagram

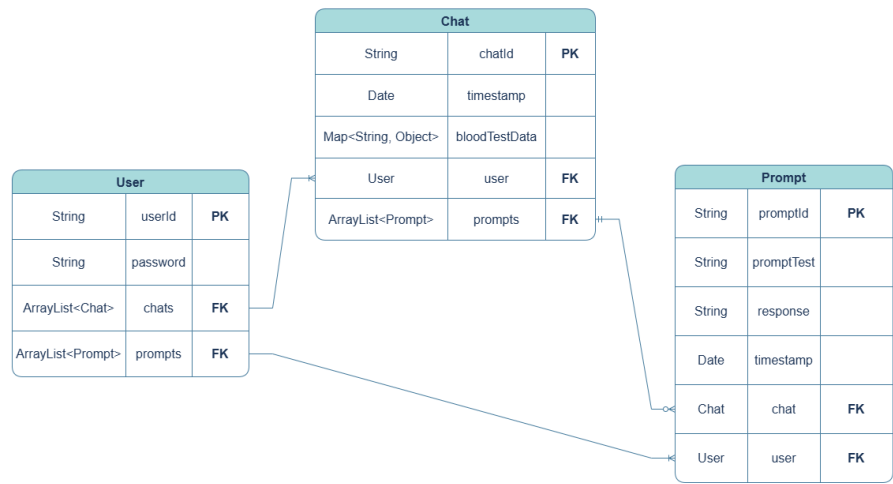


Figure 4.3: ER Diagram

4.4 Use Case Diagram

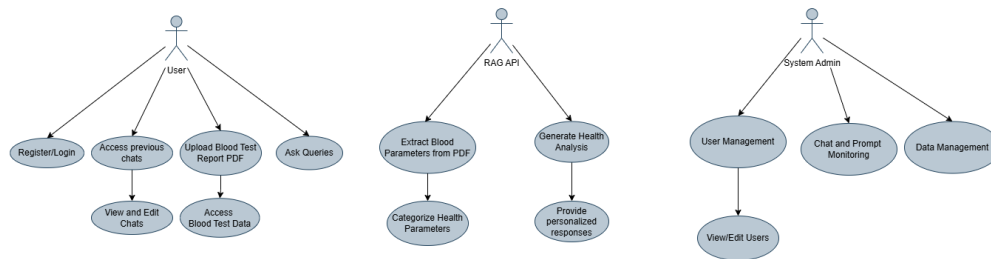


Figure 4.4: Use Case Diagram

4.5 Sequence Diagram

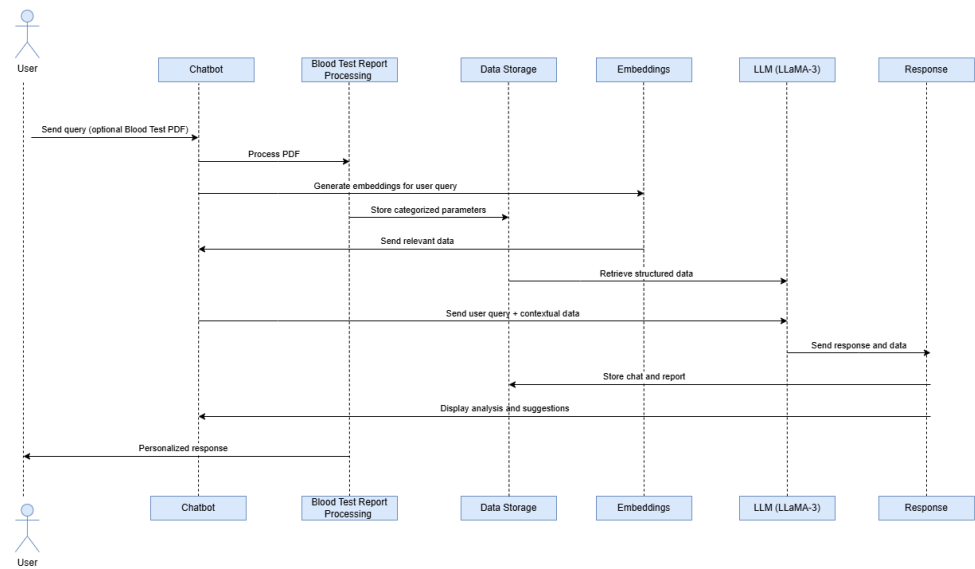


Figure 4.5: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Domain Research and Data Preparation:** Estimated 2 months for conducting in-depth research into metabolic syndrome and its key health parameters, including glucose levels, cholesterol, and blood pressure. This phase includes gathering data from reliable sources such as medical books, YouTube videos, and research papers. The research also involves understanding the latest advancements in AI and health-care technology to identify the most suitable approach for the system's development.
- **Model Development and Training:** Estimated 3 months for developing the model. This phase includes selecting the appropriate machine learning model (Llama-3 8B), setting up the Retrieval-Augmented Generation (RAG) pipeline, and training the model with domain-specific data. The focus will be on ensuring the model generates accurate and contextually relevant responses for interpreting blood test reports and providing personalized health recommendations.
- **System Integration and Testing:** Estimated 1.5 months for integrating the model into the overall system and performing rigorous testing. This phase includes connecting the model to the user interface built with React.js, integrating it with the backend system powered by Spring Boot, and linking the MongoDB database for data storage. Testing will focus on functionality, accuracy, user experience, and robustness of the system.
- **Documentation and Review:** Estimated 0.5 months to create comprehensive project documentation. This will include the technical architecture, user manuals, and guidelines for system use and maintenance. Documentation will also cover the model's performance, training details, and any troubleshooting steps. Regular review sessions will be conducted with stakeholders and mentors to gather feedback and ensure that the system meets all required objectives.

5.1.2 Project Resources

1. **Frontend Developers:** Responsible for designing and developing the user interface where users can upload blood test reports and view personalized health suggestions. The frontend is built using React.js,

HTML, CSS, and JavaScript to provide a responsive and intuitive experience for seamless interaction with the GenAI-powered system.

2. **Backend Developers:** Tasked with developing and managing the backend infrastructure using Spring Boot. This includes handling report uploads, preprocessing inputs for the LLM, managing API communications with OpenAI and LLaMA 8B, and orchestrating interactions with the database. Backend developers also ensure secure, efficient, and scalable service logic.
3. **RAG Pipeline Engineers:** Responsible for integrating the retrieval-augmented generation (RAG) pipeline, which connects extracted report data with contextually relevant medical knowledge to generate responses using the LLaMA 8B model. This includes prompt design, managing token limits, and ensuring coherent output generation based on uploaded user data
4. **Database Technologies:** MongoDB is used to store user profiles, blood test data, system logs, and chatbot interactions. The database setup supports efficient querying and ensures that all sensitive data is handled with proper access control and encryption practices.
5. **Collaboration Tools:** Git and GitHub are used for code version control and collaborative development across the tech stack. Google Drive is employed for organizing research materials and documentation. Communication and sync-up meetings are facilitated via WhatsApp and Google Meet to ensure timely discussions and updates across the team.

5.2 Risk Management

Risk management is a crucial aspect of our project to ensure its smooth implementation and success. By identifying and addressing potential risks early, we aim to minimize any negative impact on the development, deployment, and functionality of the project. This proactive approach will enhance the project's reliability, security, and performance.

5.2.1 Risk Identification

Risk identification involves recognizing potential challenges or obstacles that may hinder the project's progress. In this project, the key risks identified include:

1. Technical Risks:

- Integration challenges across components: Ensuring that the React.js frontend, Spring Boot backend, and MongoDB database work seamlessly with the model to provide a smooth and responsive user experience.
- Model Query Latency and Token Limits: Using OpenAI for data extraction and LLaMA 8B for RAG responses may lead to response delays and token limit issues, especially with complex or lengthy blood test reports.
- Database performance issues: Potential slowdowns in data retrieval and storage operations, especially with large datasets and real-time updates.

2. Business Risks:

- Changing client requirements: Evolving expectations from the sponsor regarding system functionalities, medical data interpretations, or platform integrations could cause rework or scope changes.
- Data unavailability or quality issues: Lack of publicly available high-quality blood test data may hinder validation or limit the diversity of test cases used for evaluation.
- Resource Constraints: Limited budget or time constraints may restrict system scalability, limit enhancements, or delay deployment.

3. Resource Risks:

- Multitasking across technologies: Working across multiple stacks (React, Spring Boot, MongoDB) may stretch team members, leading to delays or errors.
- Overburdened team members: Tight deadlines and multiple responsibilities could lead to team burnout, impacting productivity.
- High turnover: Losing key developers or engineers could result in delays.

4. Security Risks:

- Exposure of sensitive health data: Improper handling of user-uploaded blood test reports may result in unauthorized access or privacy breaches.
- Unencrypted storage and transmission: Storing or transmitting medical data without encryption poses risks of data leaks or cyberattacks.
- Lack of compliance with healthcare regulations: Failing to adhere to data protection laws and standards (e.g., HIPAA, GDPR) could raise legal concerns.

5.2.2 Risk Analysis

After identifying the risks, we conducted a risk analysis to determine their potential impact and develop mitigation strategies:

- **Technical Risks:**

- *Mitigation:* Ensure strong integration through modular APIs and conduct full-stack testing for React.js, Spring Boot, and MongoDB. To handle model latency and token issues, implement caching mechanisms and response chunking strategies. Optimize database queries and indexing to maintain performance under high loads.

- **Business Risks:**

- *Mitigation:* Maintain regular communication with the sponsor to manage evolving requirements and set realistic expectations. Use synthetic data generation and anonymized datasets to overcome data limitations. Prioritize core features within the available budget and use agile sprints to manage time effectively.

- **Resource Risks:**

- *Mitigation:* Clearly allocate roles based on skill strengths and provide support where cross-stack work is needed. Monitor workload distribution to avoid burnout. Maintain up-to-date documentation and onboarding material to mitigate delays from team changes.

- **Security Risks:**

- *Mitigation:* Use encryption for all data at rest and in transit. Implement strict access control policies. Ensure secure handling of user health data through standard protocols.

5.2.3 Overview of Risk Mitigation, Monitoring, and Management

Effective risk management requires ongoing assessment, monitoring, and timely intervention. Below is the summary of how our project will address risks holistically:

- **Risk Mitigation:** Proactive strategies including redundancy planning, skill upskilling, and architectural robustness have been employed to reduce the likelihood of adverse events.
- **Risk Monitoring:** A continuous monitoring mechanism will be implemented to track project metrics, model performance, security breaches, and resource utilization. Automated alerts and regular status meetings will assist in staying ahead of potential issues.
- **Risk Management:** A centralized risk register will be maintained and reviewed bi-weekly. Each risk will be assigned a severity rating, a mitigation owner, and a review schedule. Risk response plans will be revisited and adjusted as the project evolves to ensure alignment with the dynamic nature of the manufacturing domain and reinforcement learning workflows.

5.3 Project Schedule

5.3.1 Project Task Set

The following set of tasks outlines the step-by-step workflow adopted for the successful implementation of the reinforcement learning-based inventory management and production scheduling framework:

- Conduct a survey to understand challenges in blood test report interpretation and the need for personalized health recommendations.
- Organize discussions with stakeholders (e.g., healthcare professionals, domain experts) to gather detailed requirements for the system's functionalities and user experience.
- Align the project goals with the sponsor's objectives, focusing on providing accurate, personalized health advice based on blood test data to help reverse or mitigate metabolic syndromes.
- Brainstorm and analyze the requirements to define the scope of functionalities for the chatbot, ensuring it addresses the specific needs of the target users.
- Document the key issues identified by stakeholders, including any challenges related to data extraction, report interpretation, and user feedback, to establish a comprehensive problem statement.
- Identify the technology stack, evaluating feasibility based on factors such as performance, scalability, and integration capabilities. The stack includes OpenAI for data extraction, LLaMA 8B for RAG-based responses, Spring Boot for backend development, React.js for frontend, and MongoDB for database storage.
- Explore the latest techniques in machine learning, natural language processing, and healthcare data analysis to identify optimal methods for blood test report interpretation and personalized recommendations.
- Allocate project tasks—including frontend development, backend development, model integration, and testing—based on team members' strengths and expertise.
- Ensure all team members are familiar with the selected tools and technologies (OpenAI, LLaMA 8B, Spring Boot, React.js, MongoDB), providing necessary training where needed.

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

- Design interactive user interfaces tailored to the needs of healthcare professionals and end-users (patients).
- Present and seek approval for the proposed UI/UX designs from stakeholders to ensure they meet expectations regarding usability and accessibility.
- Simultaneously develop user interfaces and functional modules while integrating the model to ensure seamless interaction between the chatbot and the backend systems.
- Research and select the most suitable machine learning models for extracting and interpreting blood test data and generating personalized health recommendations.
- Begin concurrent development of frontend and backend functionalities, ensuring smooth integration with the LLaMA 8B model for real-time responses.
- Develop, test, and refine the chatbot system based on user feedback and real-world data, ensuring that it provides accurate and relevant health recommendations.
- Iterate on the chatbot's performance, improving its accuracy in interpreting blood test results and suggesting dietary and lifestyle changes based on feedback from stakeholders and users.
- Maintain detailed project documentation, including source code explanations, model architecture descriptions, API references, and user manuals, to support future scalability and maintenance.
- Document and submit the final process and codebase as part of the complete project deliverable.

5.3.2 Task Network

The Task Network outlines the sequence and interdependencies of tasks required for the successful development of the healthcare chatbot system. This roadmap ensures the timely execution of each task, guiding the project from the initial research phase through to model development, system integration, and testing. Understanding the critical dependencies among tasks is crucial to avoid delays that could impact subsequent phases of the project.

- **Requirement Analysis**

- Gathered project goals and expectations from the sponsor.
- Identified user needs for interpreting blood test reports.
- Defined the scope of system functionalities (chat interface, report analysis, and personalized suggestions).

- **Research and Technology Evaluation**

- Explored existing GenAI healthcare applications and data processing methods.
- Studied architecture and capabilities of LLaMA 8B for RAG-based interaction.
- Evaluated OpenAI models for blood report data extraction.
- Assessed feasibility of using Spring Boot, React.js, and MongoDB for system integration.

- **Data Preparation**

- Curated domain-specific text from books, research papers, and video transcripts.
- Structured and cleaned the dataset for the retrieval component of the RAG pipeline.
- Generated synthetic blood test reports for testing and model prompting.

- **Model Development**

- Implemented RAG pipeline using LLaMA 8B with vector-based document retrieval.
- Integrated OpenAI model to extract and structure data from blood test reports.

- Evaluated model responses for accuracy, context, and personalization.
- **System Integration**
 - Developed frontend using **React.js** for user interaction.
 - Created backend services with **Spring Boot** to manage communication with the model and database.
 - Configured **MongoDB** for storing blood report data and chat history.
 - Ensured seamless interaction between frontend, backend, and AI components.
- **Testing and Optimization**
 - Conducted unit testing and integration testing of system modules.
 - Implemented security checks for sensitive data handling.
- **Documentation and Review**
 - Created user manual, technical documentation, and system architecture diagrams.
 - Collected feedback from mentors and revised based on suggestions.
 - Finalized report and prepared for presentation and deployment.

Each task in the task network contributes to the overall project milestones and impacts the subsequent steps. Delays in core activities, such as data collection, system integration, or testing, could affect the entire project timeline. A well-structured and monitored task set is essential for maintaining the project's flow and ensuring that all deliverables are met on schedule, facilitating smooth and efficient progress toward project completion.

5.4 Team Organization

5.4.1 Team Members and Roles

- **Arohi Karhade:** Frontend Developer, Backend Developer, Documentarian
- **Isha Verma:** System Designer, RAG Pipeline Engineer, Documentarian
- **Varad Gadoakar:** System Designer, Backend Developer, Documentarian
- **Prof. J. S. Mahajan:** Internal Guide and Mentor
- **Mr. Prashant Karhade (Avantza Technologies Pvt. Ltd.):** External Guide and Mentor

5.4.2 Management, Reporting and Communication

Management:

For effective project management of our GenAI-powered healthcare chatbot system, we adopted a combination of collaborative tools and structured workflows. Google Drive served as the central repository for all research papers, training data (such as transcripts, blood report PDFs), and documentation, ensuring accessibility and consistency across the team. Git and GitHub were used for source code management and version control, enabling multiple contributors to work concurrently and track changes efficiently.

Reporting:

We followed a weekly reporting structure to update our internal mentor about completed tasks, encountered issues, and decisions taken regarding system architecture and LLM integration. We also conducted bi-weekly sessions with our external guide, where we discussed challenges in prompt engineering, data extraction from reports using OpenAI models, and system responses. These reviews helped ensure our progress aligned with the project's goal of delivering health recommendations using GenAI-powered report analysis.

Communication:

Clear and consistent communication was a cornerstone of our project workflow. In addition to scheduled meetings and updates, we maintained active group chats and email threads to ensure real-time collaboration and quick resolution of any issues or queries. This transparent communication culture helped in maintaining momentum and accountability throughout the project.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

This section outlines the key steps followed during the implementation of the system, divided into two primary components: data collection and pre-processing, and model selection. Each stage was carefully designed to ensure that the RAG-based health chatbot effectively interprets blood test data and provides personalized insights related to metabolic syndromes.

6.1.1 Data Collection and Preprocessing

To build a reliable and domain-specific knowledge base for the RAG (Retrieval-Augmented Generation) model, we curated data from three major sources:

- **Medical Books (PDFs):** A selection of authoritative books and guides focused on metabolic syndromes, natural health practices, and evidence-based treatment methods were collected. These resources provide a solid foundation of structured and validated medical information.
- **YouTube Videos:** We incorporated a set of high-quality health and wellness videos where qualified medical professionals and nutritionists discuss natural approaches to reducing and reversing metabolic syndromes. These videos often include practical tips, case studies, and success stories.
- **Parameter Document:** A structured document defining 33 key blood test parameters relevant to diagnosing and managing metabolic syndromes (such as fasting glucose, HDL, LDL, triglycerides, etc.) was included. It serves as a reference standard for interpreting values from blood reports.

6.1.2 Model Selection

We evaluated multiple large language models (LLMs) to determine the most suitable choice for our application, which focuses on interpreting medical data and generating actionable health insights. The models considered included:

- **OpenAI's GPT-based Models:** Known for their strong performance and commercial support, but restricted by cost and API limitations for fine-tuning and local deployment.
- **Deepseek Model:** A specialized open-source model with promising performance, though it had limited documentation and integration support compared to mainstream alternatives.

- **Meta’s LLaMA-3 (8B Parameters):** An open-source model offering a strong balance between performance, flexibility, and accessibility.

After comparative analysis, **LLaMA-3 8B** was selected for the following reasons:

- **Open-Source Flexibility:** LLaMA-3’s open-source nature allows for easy customization and integration with our RAG pipeline. We were able to fine-tune the retrieval and response handling to align with our use case.
- **Computational Efficiency:** The 8-billion parameter version offers powerful performance without requiring enterprise-grade hardware, making it ideal for deployment using APIs like Ollama, which supports lightweight inference.
- **Community and Tooling Support:** LLaMA-3 enjoys strong community support, comprehensive documentation, and compatibility with popular libraries such as Hugging Face Transformers, LangChain, and Ollama. These resources simplified development and accelerated deployment.
- **Balanced Accuracy and Cost:** While being cost-effective due to its open availability, the model maintains high-quality output, particularly when used in a RAG setting where the quality of retrieved context boosts answer precision.

This combination of high-quality contextual data and a capable, efficient language model forms the core of the intelligent chatbot system designed to provide health recommendations based on blood test data.

6.1.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is the process of optimizing the output of a large language model (LLM) by referencing an authoritative knowledge base external to its training data prior to generating a response. This approach enhances the performance of LLMs for specific use cases, increasing both accuracy and relevancy. RAG extends the capabilities of general-purpose LLMs to specific domains or an organization’s internal knowledge base without requiring retraining of the model.

Traditional LLMs rely solely on static pre-trained knowledge, which can often be generalized or outdated. RAG introduces dynamic retrieval of

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

relevant information by leveraging vector search over stored embeddings. The RAG pipeline consists of two primary components: the **retrieval pipeline** and the **generation pipeline**.

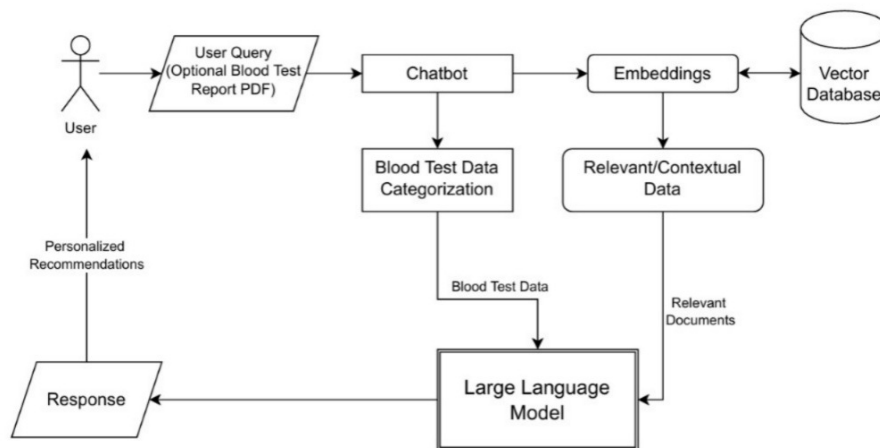


Figure 6.1: RAG Pipeline

6.1.3.1 Building the Vectorized Knowledge Base

The raw text extracted from diverse sources (books, transcripts, documents) is converted into numerical embeddings. First, the long-form text is divided into semantically coherent chunks using the **FastEmbedEmbeddings** module. This chunking process ensures that each segment carries enough medical context for query resolution and improves retrieval granularity.

We used LangChain to implement semantic-aware chunking, which avoids breaking important medical concepts across segments. The resulting text chunks are then transformed into dense vector embeddings—mathematical representations that capture semantic meaning. These document embeddings are stored in **ChromaDB**, a high-performance vector store that enables persistent and efficient similarity searches.

6.1.3.2 Retrieval Pipeline

When a user submits a query, it is also transformed into a vector embedding using the same embedding model. This ensures compatibility between query and document representations. A semantic similarity search is then performed using either cosine similarity or inner product comparison.

Each document chunk is assigned a similarity score, and the most relevant top- k (where $k = 3$) results are selected and appended as context to the original query. Low-relevance documents are filtered out using a threshold-based mechanism to maintain the quality of context provided to the generation model.

6.1.3.3 Generation Pipeline

The selected and concatenated chunks are combined with the user's query to form a *context-augmented prompt*. This prompt is then processed by the LLM, which generates a response that is both informative and domain-specific.

To promote transparency and trust, the system includes references alongside the response. These may include book titles, YouTube video links, or other source documents. For video sources, specific links are provided to allow users to verify the information. If the system fails to retrieve relevant context, it defaults to generating a generalized response, avoiding hallucinations or misleading answers.

The model has been specifically prompted to deliver clear and easy-to-understand explanations of blood test results, along with actionable dietary and lifestyle suggestions. These recommendations aim to help users improve their health and potentially reverse or prevent the onset of metabolic disorders.

6.1.4 System Integration and Implementation

To transform the RAG-based health chatbot into a fully functional web application, we developed a complete system architecture consisting of three main components: the frontend (React.js), the backend (Spring Boot), and the AI layer (RAG Model with LLaMA-3 via Ollama API). Each component plays a crucial role in delivering a seamless and interactive user experience.

6.1.4.1 Frontend Development (React.js)

The frontend was built using **React.js**, a modern JavaScript library for creating dynamic and responsive web interfaces. It serves as the primary interaction layer between users and the system.

Key functionalities implemented:

- **User Authentication:** Users can register and log in securely to access their personalized dashboard.

- **Report Upload:** A user-friendly upload interface allows users to submit blood test reports in formats such as PDF or image files.
- **Chat Interface:** An integrated chatbot interface allows users to ask questions, receive report analysis, and view recommendations.
- **Result Display:** Users receive real-time responses with health insights, dietary advice, and lifestyle tips, all displayed in a readable format.

6.1.4.2 Backend Development (Spring Boot)

The backend of the system was developed using **Spring Boot**, a robust Java framework designed to handle server-side logic and API communication.

Key responsibilities of the backend:

- **API Handling:** Handles REST API requests from the frontend and routes them to the appropriate services.
- **User Data Management:** Manages user profiles, session data, and report metadata in a secure and structured manner.
- **Security:** Implements authentication and authorization protocols to ensure secure access to user data.
- **Communication with AI Layer:** Sends user queries and report content to the RAG model and receives responses for delivery to the frontend.

6.1.4.3 AI Integration with RAG Model

The AI engine is powered by the **LLaMA-3 8B RAG model**, deployed using the **Ollama API**. This component is responsible for:

- Accepting input from the backend, including raw text from reports or user queries.
- Retrieving relevant information from the vectorized knowledge base via the RAG retrieval pipeline.
- Generating personalized responses using the generation pipeline of the RAG model.
- Sending structured and citation-backed responses back to the backend.

6.1.4.4 System Workflow

The complete system operates in the following sequence:

1. The user logs in and uploads a blood test report or asks a health-related query through the React interface.
2. The frontend sends this input to the Spring Boot backend.
3. The backend processes the request and forwards it to the RAG model API hosted via Ollama.
4. The AI model retrieves context, generates a response, and sends it back to the backend.
5. The backend relays the response to the frontend for display to the user.

This tightly coupled integration of frontend, backend, and AI enables a responsive, secure, and intelligent system that helps users understand their health reports, get actionable advice, and interact naturally with a specialized health chatbot.

6.2 Tools and Technologies Used

In the development of our system for analyzing blood test reports and providing personalized health insights, we employed a range of modern tools and technologies. These technologies enabled us to build a robust, scalable, and interactive system. Below is an overview of the key tools and technologies used:

6.2.1 Frontend Development

- **React.js:** The frontend of the GenAI system is built using React.js, a popular JavaScript library for creating user interfaces. React allows for the development of dynamic, responsive, and reusable UI components that enhance user experience.
- **HTML/CSS:** HTML and CSS are used for structuring and styling the web pages. CSS frameworks like Bootstrap are utilized to ensure the interface is mobile-responsive and visually appealing.
- **JavaScript:** JavaScript is used for client-side scripting to handle user interactions, manage state, and communicate with the backend via RESTful APIs.

6.2.2 Backend Development

- **Spring Boot:** The backend of the system is developed using Spring Boot, a Java-based framework that simplifies the development of robust, scalable, and production-grade applications. Spring Boot provides built-in support for REST APIs, security, and data management.
- **Java:** Java is the core programming language used to implement the business logic, manage data flow, and handle communication between the backend and the AI model.

6.2.3 Artificial Intelligence and Model Integration

- **Retrieval-Augmented Generation (RAG) Model:** The RAG model, particularly the Llama-3 8B model, is used to process medical data and generate actionable health insights. It combines the power of large language models with retrieval mechanisms to provide contextually relevant and accurate answers to user queries.

- **Ollama API:** Ollama API is used to support the inference process of the Llama-3 8B RAG model. It enables the seamless deployment and use of the model for generating responses based on user queries and health data.
- **Langchain:** Langchain is used to facilitate integration between the RAG model and the application, managing the flow of data between the backend and the AI model.
- **ChromaDB:** ChromaDB is used as a vector store to store document embeddings, enabling efficient similarity-based search for retrieving contextually relevant information to augment the responses from the RAG model.
- **FastEmbedEmbeddings:** This module is used to vectorize the raw text data extracted from various sources, transforming it into numerical embeddings for semantic similarity search.
- **FlaskAPI:** We used a Flask API to connect the frontend with our Python RAG model. We set up an /extract endpoint to handle the extraction of data from the blood test PDF. Once the data was extracted, we redirected the request to the /ask endpoint, which received the prompt and the extracted blood test data in JSON format. The API processed this data using the RAG model to generate a personalized analysis. Then, we returned the response to our Spring Boot backend, where it was stored and shown to the user.

6.2.4 Database

- **MongoDB:** MongoDB, a NoSQL database, is used to store user data, blood test reports, and analysis results. It is chosen for its flexibility in handling unstructured data and its ability to scale horizontally as the user base grows.

6.2.5 Data Processing and Natural Language Processing

- **PyMuPDF:** PyMuPDF is used for extracting text from PDFs, specifically medical books and reports, which are then preprocessed and stored for further analysis.

- **OpenAI Whisper:** The Whisper module is used for transcribing YouTube videos into text, making it easy to extract relevant medical information from spoken content.

6.2.6 Version Control

- **Git:** Git is used for version control to manage the source code, track changes, and collaborate effectively with the development team. GitHub is used to host the code repository.

These tools and technologies were carefully selected to ensure that the system is capable of handling complex tasks, such as processing medical data, interacting with users, and generating personalized health insights. The combination of powerful backend systems, modern frontend technologies, and advanced AI capabilities makes the system robust, efficient, and scalable.

6.3 Algorithms Used

To optimize scheduling and inventory decisions in a dynamic manufacturing environment, we experimented with and implemented various reinforcement learning algorithms. These were selected based on problem complexity, scalability, and decision accuracy.

6.3.1 Embedding Algorithm: FastEmbedEmbeddings

This algorithm converts text into dense vectors that capture semantic meaning using transformer-based models. These embeddings enable efficient similarity comparison between user queries and stored documents.

6.3.2 Vector Similarity Search

Chroma uses cosine similarity to find document chunks most relevant to the user's query based on their vector embeddings. It supports filtering by a similarity threshold to ensure only contextually relevant documents are passed to the LLM.

6.3.3 Recursive Character Text Splitter

This splitter breaks large texts into smaller overlapping chunks based on character count and separator rules. Smaller chunks improve retrieval accuracy and avoid exceeding token limits in LLM input.

6.3.4 LLM Inference (ChatOllama using LLaMA3)

The model generates human-like responses by predicting the next word in context using a transformer architecture. It processes both the user query and retrieved document chunks to deliver informed, coherent answers.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

To ensure the reliability and robustness of the developed system, multiple levels of software testing were conducted. The testing strategy included both functional and non-functional testing methodologies, aimed at validating the accuracy, usability, and performance of the system modules.

7.1.1 Unit Testing

Unit Testing was performed to validate individual modules such as the PDF extraction logic, parameter categorization, and Spring Boot service methods. It ensured that each component—such as JSON parsing, prompt handling, or health analysis generation—works correctly before integrating into the full pipeline.

7.1.2 Integration Testing

Integration Testing was conducted to verify the interaction between different modules, such as the React frontend, Spring Boot backend, and Python RAG API. It ensured that data flowed correctly—from PDF upload to health analysis response—across the entire system.

7.1.3 System Testing

System Testing was carried out to evaluate the entire application as a complete and integrated system. It ensured that all components—from user registration to PDF analysis and chat history retrieval—worked together seamlessly to meet functional requirements.

7.1.4 Performance Testing

Performance Testing was performed to assess the responsiveness and stability of the GenAI system under various workloads. It ensured that operations like PDF processing, API calls, and data retrieval from MongoDB remained efficient even with increased user activity.

7.1.5 User Acceptance Testing (UAT)

User Acceptance Testing (UAT) was conducted with end users to validate that the GenAI system met their expectations and functional requirements.

Users tested key features like uploading PDFs, receiving health analysis, and viewing past chats to ensure a smooth and intuitive experience.

7.2 Test Cases and Results

We performed a manual analysis of the AI responses. We scored each response on four dimensions—Correctness, Clarity, Helpfulness, and Completeness—between 1 and 100. Table 7.2 shows the values of the metrics. Figure 7.1 shows the pictorial representation of the metrics values. The evaluation prompt was standardized and crafted to mimic expert review. Human monitoring guaranteed consistency and reliability. The approach is representative of a semi-automated, expert-in-the-loop assessment strategy, striking a balance between scalability and clinical understanding.

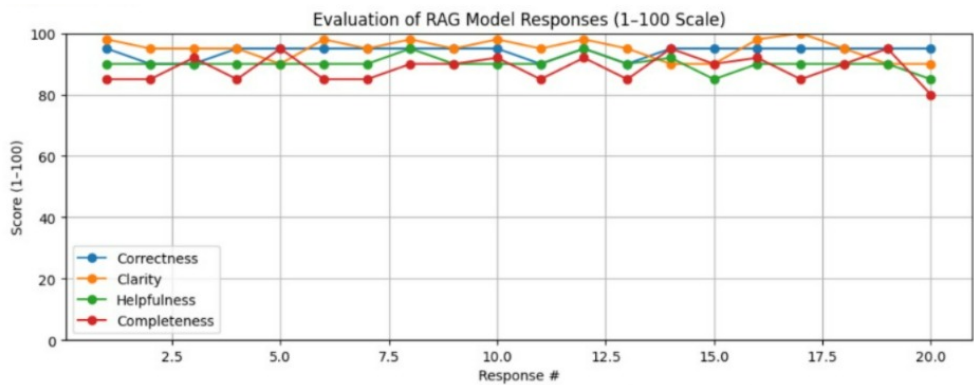


Figure 7.1: Result Graph

Test Case Description	Input Data	Expected Output	Status
Upload valid PDF blood report	Valid PDF with standard blood parameters	JSON with extracted values and their categories	Pass
Upload invalid file format	File with .txt or .jpg extension	Error message: "Invalid file format. Please upload a PDF."	Pass

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

Missing values in blood test report	PDF with missing Hemoglobin and HDL values	JSON with null or “Missing” entries in those fields	Pass
Generate analysis for normal values	Categorized JSON with all values in normal range	Response: Gave a generalized response to maintain good health	Pass
Generate analysis for metabolic syndrome indicators	Categorized JSON with high triglycerides, low HDL, and high fasting glucose	Response identifies Metabolic Syndrome risk and provides dietary/lifestyle suggestions	Pass
Save and retrieve chat session	Prompt + response stored with chat ID and user ID	Chat history retrieved successfully under the same session	Pass
Handle multi-turn conversation in one session	Multiple prompts under the same chat ID	Contextual responses returned for each prompt, maintaining conversation history	Pass
Upload PDF and prompt together	PDF + prompt: “Give me analysis for this report.”	Combined processing and contextual response from RAG model	Pass
Handle empty prompt with valid PDF	Blank prompt + valid PDF	Response: Personalized dietary and lifestyle recommendations	Pass
Large file handling	PDF with 10+ pages of blood test data	Proper extraction, categorization, and generation of complete analysis	Pass

Table 7.1: Test Case Table

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

Response #	Correctness	Clarity	Helpfulness	Completeness
1	95	98	90	85
2	90	95	90	85
3	90	95	90	92
4	95	95	90	85
5	95	90	90	95
6	95	98	90	85
7	95	95	90	85
8	95	98	95	90
9	95	95	90	90
10	95	98	90	92
11	90	95	90	85
12	95	98	95	92
13	90	95	90	85
14	95	90	92	95
15	95	90	85	90
16	95	98	90	92
17	95	100	90	85
18	95	95	90	90
19	95	90	90	95
20	95	90	85	80

Figure 7.2: Results Table

CHAPTER 8

RESULTS

8.1 Outcomes

The project successfully delivered a working prototype of a health advisory system capable of analyzing user-uploaded blood test reports and generating personalized suggestions using a combination of OpenAI for report data extraction and the LLaMA 8B model for reasoning and recommendations. The results achieved through the development and integration phases are as follows:

8.1.1 Report Upload and Parsing

Users were able to upload blood test reports in PDF format through the React.js frontend. Upon upload, the system used OpenAI's API to extract relevant textual and tabular information from the reports. The extracted data was parsed on the backend to isolate individual parameters and their values accurately. This phase ensured high precision in mapping blood parameters, eliminating inconsistencies that may have arisen due to varied report formats.

8.1.2 Parameter Evaluation and Categorization

After extraction, each blood test parameter was compared against carefully constructed reference ranges stored in the backend. The system categorized values as:

- **Optimal** – within the healthy range
- **Borderline** – slightly outside the healthy range
- **Critical** – significantly above or below normal

This structured categorization helped standardize the input for the LLM and served as a basis for determining the level of advice to be provided.

8.1.3 LLM Response Generation

The categorized results were sent to a retrieval-augmented generation (RAG) pipeline using the LLaMA 8B model. Without fine-tuning, the model was instructed through well-engineered prompts and context retrieved from a custom-built medical knowledge base. The responses generated included:

- Explanations of abnormal parameters

- Personalized dietary and lifestyle changes
- Preventive suggestions for avoiding metabolic syndrome

The LLM successfully produced user-friendly, context-specific feedback for a variety of blood test reports.

8.1.4 End-to-End System Integration

The integration across the React frontend, Spring Boot backend, MongoDB database, and the LLM API was successfully implemented. The system flow—from uploading a report to receiving suggestions—was streamlined and bug-free after several iterations. Database interactions were optimized to handle report metadata, extracted parameters, and response storage efficiently.

8.1.5 User Interface and Experience

The frontend interface was designed to be minimalistic and user-friendly. It provided users with:

- A clear upload option for blood reports
- A categorized view of test results (optimal/borderline/critical)
- Text-based health suggestions from the LLM
- Visual indicators for parameters needing urgent attention

Feedback from test users highlighted that the interface was intuitive and the results were presented in a digestible manner, even for non-technical users.

8.1.6 Stakeholder Validation and Feedback

After the system was fully integrated and functional, a demonstration was conducted for key stakeholders, including project sponsors and subject matter experts in healthcare and technology. The stakeholders reviewed the system based on criteria such as accuracy of analysis, relevance of recommendations, ease of use, and scalability.

- **Accuracy:** Stakeholders confirmed that the parameter extraction and categorization closely aligned with standard clinical practices.

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

- **Relevance:** The LLM-generated suggestions were considered contextually appropriate and valuable for users aiming to manage or reverse metabolic syndrome.
- **Usability:** The user interface was praised for its clarity, responsiveness, and accessibility for users with minimal technical background.
- **Suggestions:** Stakeholders recommended the inclusion of multilingual support and longitudinal tracking for users with multiple reports.

The validation process helped refine the final system and reinforced its potential for real-world deployment in preventive healthcare applications.

8.2 Screenshots

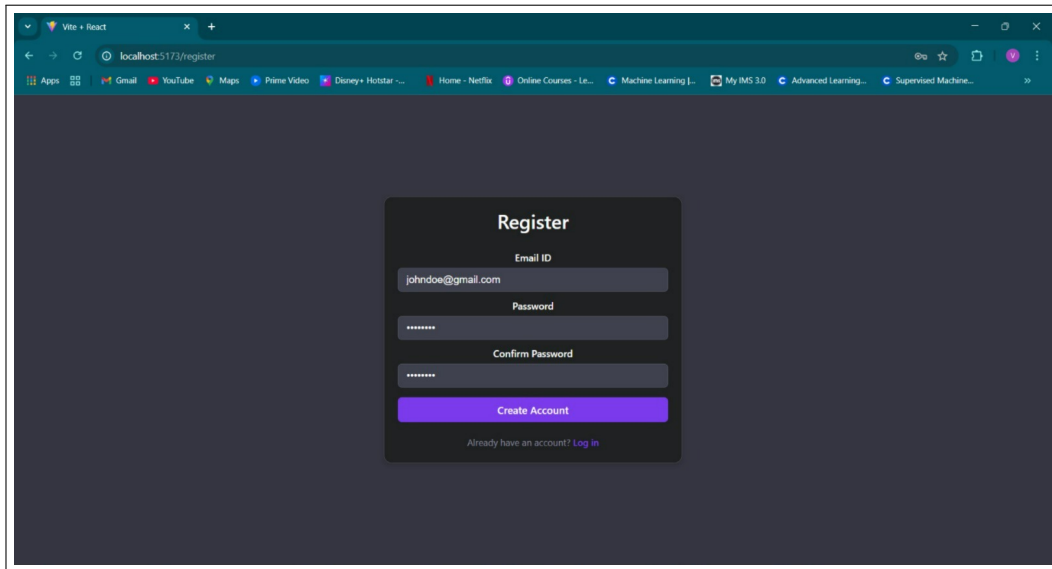


Figure 8.1: Sign Up Page

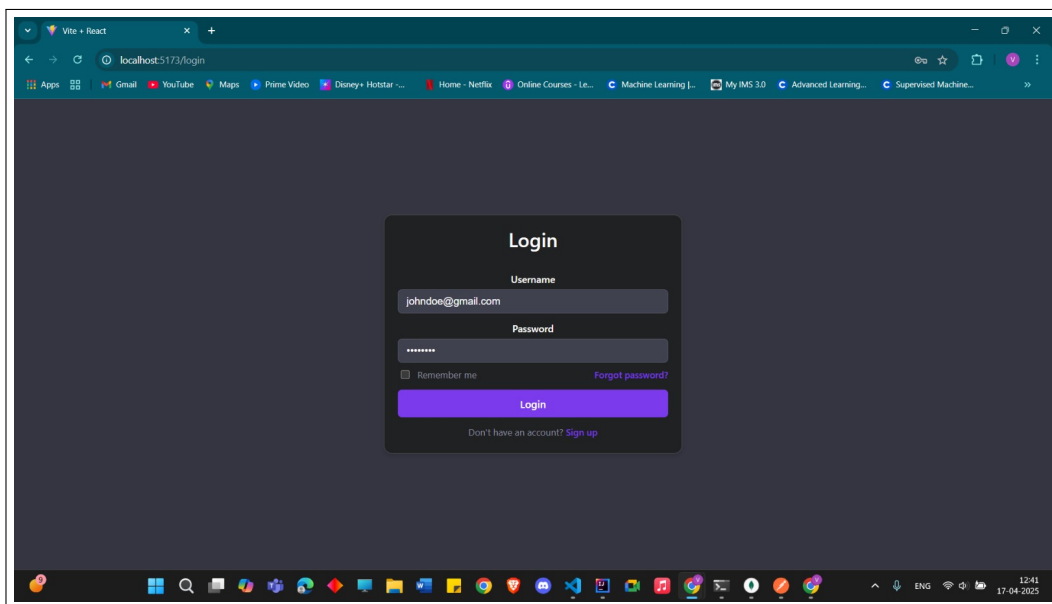


Figure 8.2: Login

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

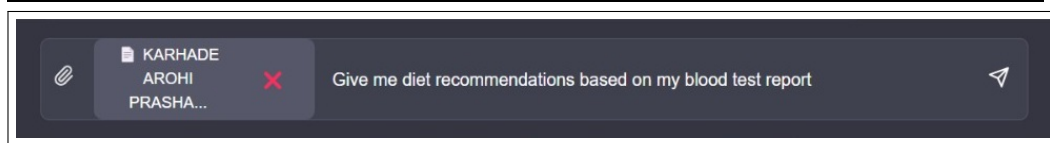


Figure 8.3: User Prompt

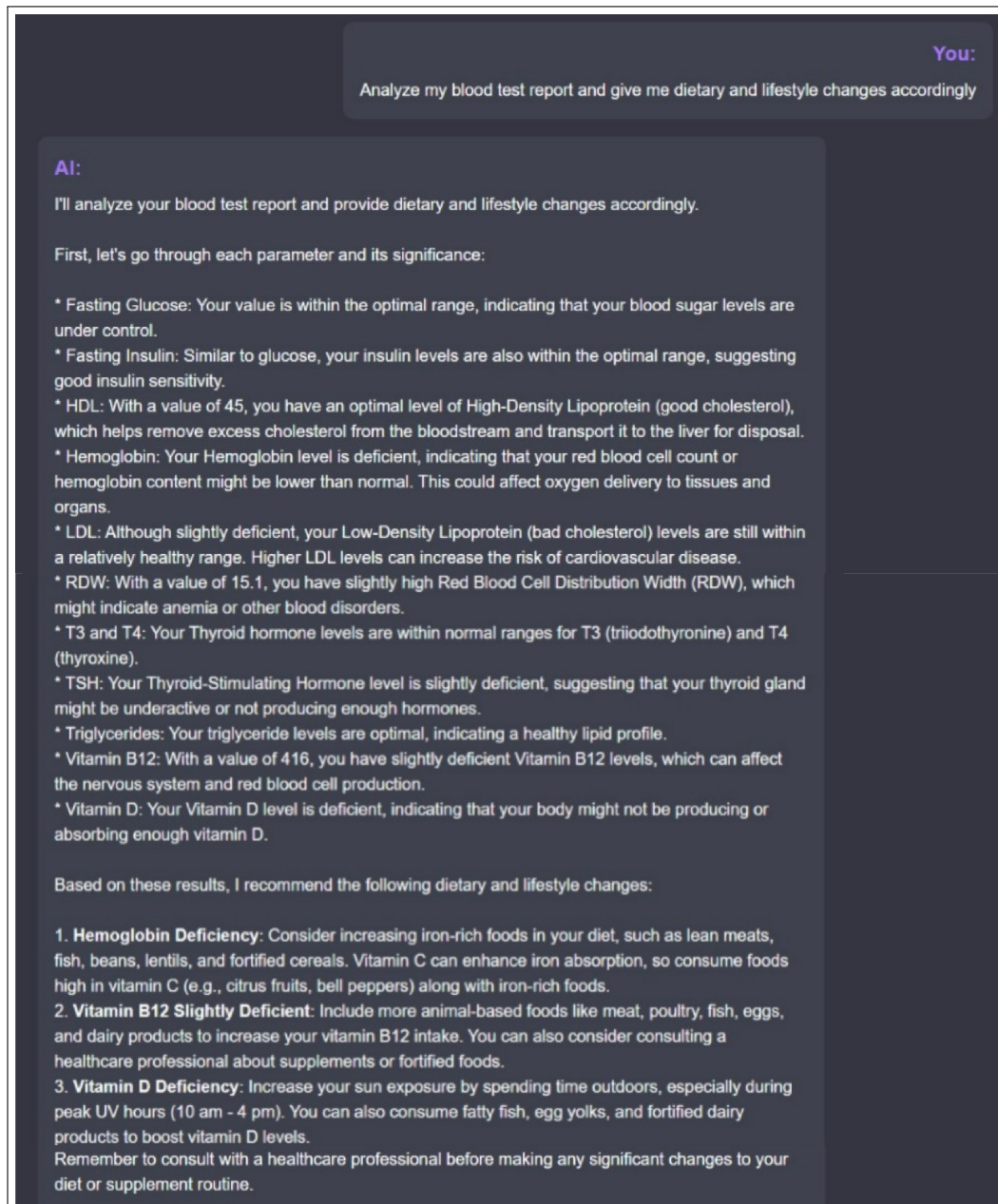


Figure 8.4: Prompt and Response

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

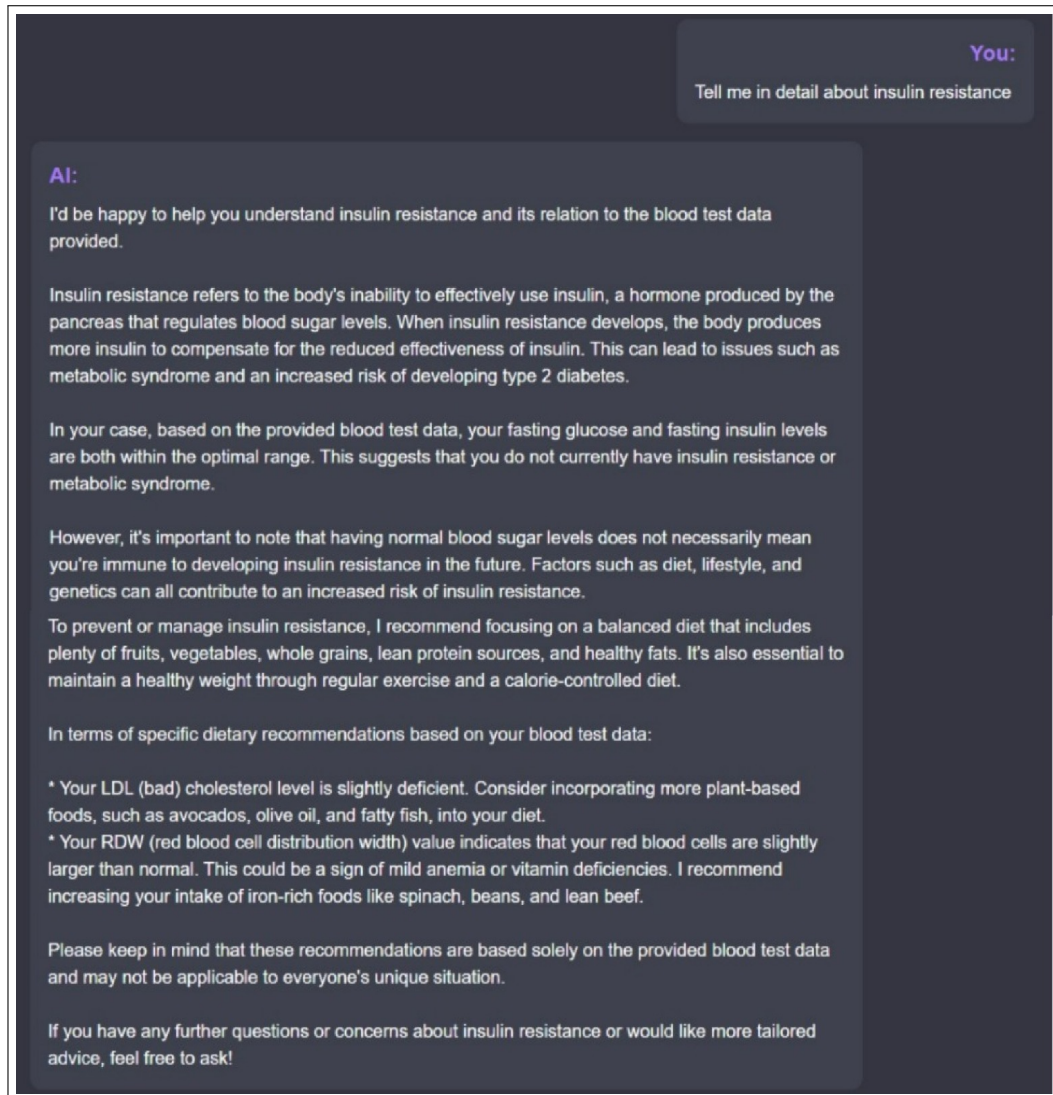


Figure 8.5: Prompt and Response

CHAPTER 9

CONCLUSION

9.1 Conclusion

This project successfully delivers a Retrieval-Augmented Generation (RAG)-based GenAI system that simplifies the interpretation of blood test reports for non-medical individuals, with a particular focus on reversing and managing Metabolic Syndrome. The system extracts structured data from PDF reports, analyzes key parameters, and generates personalized, easy-to-understand recommendations.

By combining the power of React.js for a responsive user interface, Spring Boot for backend orchestration, MongoDB for efficient data storage, and LLM-based Retrieval-Augmented Generation (RAG) for contextual recommendations, the solution demonstrated practical applicability in preventive healthcare.

More importantly, the project demonstrates how GenAI can bridge the gap between complex medical data and meaningful, personalized insights. By providing clear explanations and tailored lifestyle guidance, the system empowers individuals to take charge of their own health without the need for constant professional consultation.

In conclusion, this RAG-driven architecture presents a valuable solution for democratizing access to health information, making it easier for people to understand their blood test results, detect potential risks early, and take proactive steps toward better health outcomes.

9.2 Future Work

The future scope of this project focuses on expanding the system's capabilities to enhance user experience, accessibility, and personalization. Several key areas for improvement and growth include:

- **Multilingual Support:** Expanding the system's reach by incorporating multilingual support will allow users from diverse linguistic backgrounds to easily interpret their health data. This will enable the platform to cater to a global audience, ensuring healthcare insights are accessible to a wider range of users, regardless of their language proficiency. Such scalability can make the system applicable to multiple regions, promoting better health management worldwide.
- **Personalization:** Building on the core functionality, the system will be enhanced to analyze users' historical blood test data, allowing it to offer more personalized and predictive insights. By tracking trends over time, the system can detect early signs of health issues, including metabolic syndrome, and generate more tailored recommendations based on individual health progress. This will enable proactive, data-driven decisions to be made, empowering users to take control of their long-term health.
- **Speech Recognition:** The integration of speech recognition technology will further improve the system's accessibility. A hands-free, voice-activated interface will allow users to interact with the system in a more natural way, especially those who may find navigating the interface challenging. This addition would make the platform even more user-friendly, particularly for individuals with disabilities or those in hands-busy environments.
- **Dynamic Knowledge Base Updates (RAG):** One of the most powerful aspects of the system lies in its use of Retrieval-Augmented Generation (RAG) to provide contextually relevant and up-to-date recommendations. In the future, the knowledge base supporting the system can be dynamically updated through the integration of real-time healthcare research, clinical guidelines, and user-generated data. By continuously feeding new medical literature, health studies, and relevant research into the retrieval system, the AI can ensure that its recommendations reflect the most current and accurate medical knowledge.

9.3 Applications

The system developed in this project has a wide range of applications in the healthcare domain, particularly in areas that focus on improving accessibility, empowering patients, and supporting preventive healthcare strategies. By leveraging advanced AI techniques, the system can provide personalized insights and recommendations, making it a valuable tool for both individual users and healthcare providers. Below are the key applications of the system:

- **Preventive Healthcare:** The system can help detect early warning signs of metabolic syndromes by analyzing blood test reports, enabling timely interventions and preventive measures.
- **Remote Healthcare:** Useful for patients in rural or underserved areas, where access to healthcare professionals is limited, providing critical health insights without the need for physical consultations.
- **Patient Self-Monitoring:** Empowers users to understand their blood test results and track their health over time, reducing the need for constant visits to healthcare professionals.
- **Personalized Healthcare Platforms:** The system offers tailored lifestyle and dietary recommendations, based on individual health data, promoting personalized wellness plans.
- **Health Data Integration:** The system can be integrated into electronic health record (EHR) systems, enhancing healthcare providers' ability to make data-driven decisions by providing personalized health insights from blood test reports.

CHAPTER 10

APPENDIX

Appendix A: Problem Statement Feasibility Assessment

Problem Statement Feasibility Assessment using Satisfiability Analysis and Computational Complexity

1. Problem Complexity Overview: The system analyzes blood test reports to detect metabolic risks and generate personalized health recommendations. This involves interpreting multiple medical parameters, cross-referencing them with current medical knowledge, and generating meaningful responses using Retrieval-Augmented Generation (RAG). Such a multi-step pipeline includes data parsing, logical constraint checking, and real-time knowledge retrieval, making the computational assessment essential.

2. Satisfiability Analysis: Each blood test parameter and its classification (e.g., low, normal, high) can be treated as a Boolean variable. The system applies logical constraints derived from medical knowledge to determine valid combinations. This can be mapped to a Boolean SAT (Satisfiability) problem, where each logical clause represents a health-related constraint. Satisfiability solvers can be employed to check if a user's test results satisfy the necessary conditions for health optimization.

3. Computational Complexity Classification:

- **P-Class:** Basic operations such as comparing test values to thresholds or retrieving static recommendations can be executed in polynomial time.
- **NP-Complete:** The personalization task—optimizing recommendations from a set of interventions based on multiple test results—resembles the *Subset Sum Problem*, which is NP-Complete. Mapping test results to optimal lifestyle or dietary changes involves combinatorial search with constraints.
- **NP-Hard:** The dynamic integration of new medical research into the RAG pipeline increases complexity. As the system continuously ingests and retrieves from a growing knowledge base, ensuring real-time relevance and accuracy can resemble NP-Hard optimization problems, particularly when merging conflicting or large-scale data sources.

4. Mathematical Models Used:

- **Graph Theory:** Health parameters and their interdependencies can be modeled as graphs, enabling the use of traversal and optimization algorithms to understand complex relationships (e.g., cholesterol-glucose correlation).
- **Linear Programming:** Optimization models can be used to recommend lifestyle changes under constraints (e.g., diet under caloric limits). Decision variables are formulated to maximize health benefit.
- **Probabilistic Models:** Future state predictions (e.g., risk of diabetes) may be modeled using Markov Decision Processes (MDPs), which take into account probabilistic transitions and expected long-term rewards.

5. Conclusion: The project incorporates components that fall under P, NP-Complete, and NP-Hard classifications. While parts like individual parameter checks are simple (P), optimizing recommendations and updating the RAG knowledge base introduce significant computational complexity. Despite this, with modern AI, heuristics, and cloud computing, the system remains feasible and scalable for real-world deployment.

10.1 Appendix B

10.1.1 Paper Publication

The research and development work from this project has culminated in a paper publication, successfully accepted and published in a journal.

- **Paper Title:** Using GenAI To Generate Blood Test Report Analysis To Reverse Metabolic Syndrome
- **Paper ID:** PIJET-06
- **Journal:** PICT's International Journal of Engineering and Technology (PIJET)
- **Volume and Issue:** Volume-2, Issue-2, June 2025
- **Status:** Paper Published and accessible online

10.1.2 Certificates



Figure 10.1: Certificates of Participation in Impetus and Concepts '25

10.2 Appendix C

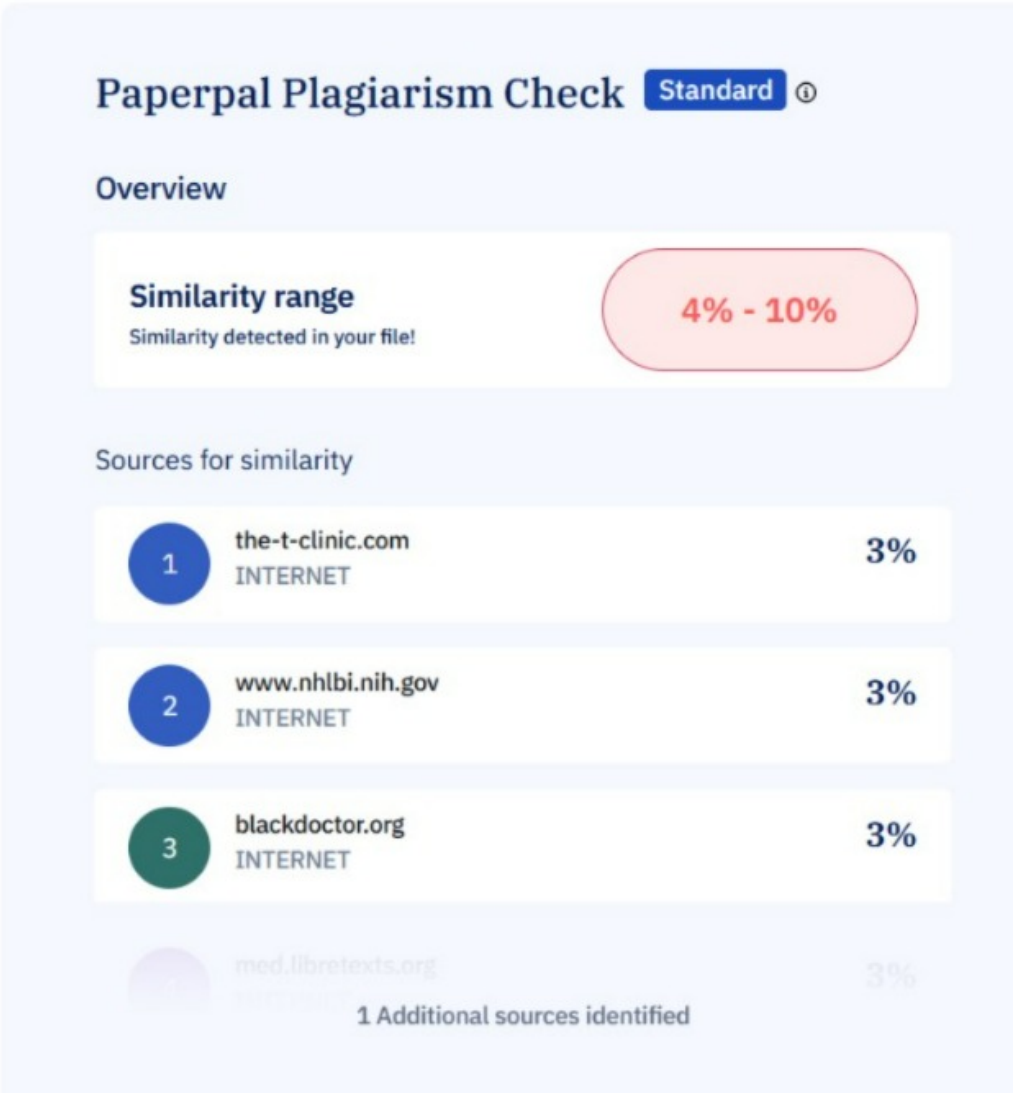


Figure 10.2: Plagiarism report

CHAPTER 11

REFERENCES

- [1] Leluc, R  mi & Kadoche, Elie & Bertoncello, Antoine & Gourv  nec, S  bastien. (2023). MARLIM: Multi-Agent Reinforcement Learning for Inventory Management. 10.48550/arXiv.2308.01649.
- [2] Muller, Arthur & Grumbach, Felix & Sabatelli, Matthia. (2024). Smaller Batches, Bigger Gains? Investigating the Impact of Batch Sizes on Reinforcement Learning Based Real-World Production Scheduling. 10.48550/arXiv.2406.02294.
- [3] Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, Dennis J. Zhang (2022) Can Deep Reinforcement Learning Improve Inventory Management? Performance on Lost Sales, Dual-Sourcing, and Multi-Echelon Problems. *Manufacturing & Service Operations Management* 24(3):1349-1368. <https://doi.org/10.1287/msom.2021.1064>
- [4] Munyaka Baraka, Jean-Claude & Yadavalli, Sarma. (2022). Inventory management concepts and implementations: a systematic review. *South African Journal of Industrial Engineering*. Vol 33, No 2. 15-36. 10.7166/33-2-2527.
- [5] Modrak, V., Sudhakarapandian, R., Balamurugan, A., & Soltysova, Z. (2024). A Review on Reinforcement Learning in Production Scheduling: An Inferential Perspective. *Algorithms*, 17(8), 343. <https://doi.org/10.3390/a17080343>.
- [6] Sutton, R.S., & Barto, A.G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [7] Kaelbling, L.P., Littman, M.L., & Moore, A.W. (1996). Reinforcement Learning: A Survey. *Journal of Artificial Intelligence Research*, 4, 237–285, <https://doi.org/10.48550/arXiv.cs/9605103>
- [8] Szepesv  ri, C. (2010). *Algorithms for Reinforcement Learning*. Morgan & Claypool Publishers.
- [9] Kuhnle, A., Kaiser, J.P., Thei  , F. et al. (2021). Designing an adaptive production control system using reinforcement learning. *Journal of Intelligent Manufacturing*, 32, 855–876. <https://doi.org/10.1007/s10845-020-01612-y>
- [10] Paraschos, P.D., Koulinas, G.K. & Koulouriotis, D.E. (2024). A reinforcement learning/ad-hoc planning and scheduling mechanism for flexible and sustainable manufacturing systems. *Flexible Services and Manufacturing Journal*, 36, 714–736. <https://doi.org/10.1007/s10696-023-09496-9>

- [11] Thürrer, M., Fernandes, N., & Stevenson, M. (2022). Production planning and control in multi-stage assembly systems: An assessment of Kanban, MRP, OPT (DBR) and DDMRP by simulation. *International Journal of Production Research*, 60(3), 1036–1050.
- [12] Velasco Acosta, A. P., Mascle, C., & Baptiste, P. (2020). Applicability of demand-driven MRP in a complex manufacturing environment. *International Journal of Production Research*, 58(14), 4233–4245.
- [13] Shahbazi et al. (2023). Metabolic syndrome prediction using non-invasive and dietary parameters: A machine learning approach. *Journal of Translational Medicine*.
- [14] Aguilera et al. (2023). Leveraging Machine Learning and Genetic Risk Scores for the Prediction of Metabolic Syndrome in Children with Obesity. *Proceedings, MDPI*.
- [15] B. A. Research, “Evaluating the feasibility of ChatGPT in healthcare: An analysis of multiple clinical and research scenarios,” *Journal of Medical Internet Research*, vol. 25, p. e40023, 2023.
- [16] González et al. (2022). Predicting Metabolic Syndrome Risks Using Machine Learning Techniques. *IEEE Journal of Biomedical and Health Informatics*
- [17] T.-Y. Liu, S. Zhang, H. Poon, T. Qin, R. Luo, L. Sun, Y. Xia, and, “BioGPT: generative pre-trained transformer for biomedical text generation and mining,” *Briefings in Bioinformatics*, vol. 23, no. 6, p. bbac409, 09 2022. [Online]. Available: <https://doi.org/10.1093/bib/bbac409>
- [18] T. Zhu, A. Chen, and L. Liu “Advancing the democratization of generative artificial intelligence in healthcare: a narrative review,” *Journal of Hospital Management and Health Policy*, vol. 8, no. 6, 2024. [Online]. Available: <https://jhmhp.amegroups.org/article/view/8842/pdf>
- [19] G. Cloud, “Sharing Google Med-Palm 2: Medical large language model,” 2023. [Online]. Available: <https://cloud.google.com/blog/topics/healthcare-life-sciences/sharing-google-med-palm-2-medical-large-language-model>
- [20] M. G. Saklayen, “The global epidemic of the metabolic syndrome,” *Current Hypertension Reports*, vol. 20, no. 2, p. 12, 2018. [Online]. Available: <https://doi.org/10.1007/s11906-018-0812-z>

Using Gen-AI to Generate Blood Test Analysis to Reverse Metabolic Syndrome

Abbreviation	Full Form
AI	Artificial Intelligence
GenAI	Generative Artificial Intelligence
PDF	Portable Document Format
HDL	High-Density Lipoprotein
LDL	Low-Density Lipoprotein
API	Application Programming Interface
JSON	JavaScript Object Notation
RAG	Retrieval-Augmented Generation
UI	User Interface
DB	Database
LLM	Large Language Model
ML	Machine Learning
HTTP	HyperText Transfer Protocol

LIST OF FIGURES

Figure	Illustration	Page No.
3.1	Agile Model for SDLC	22
4.1	System Architecture	24
4.2	Workflow Diagram	27
4.3	ER Diagram	28
4.4	Use Case	29
4.5	Sequence Diagram	30
6.1	RAG Pipeline	46
7.1	Result Graph	56
7.2	Test Case Table	57
8.1	Sign Up Page	63
8.2	Login Page	63
8.3	User Prompt	64
8.4	Prompt and Response	64
8.5	Prompt and Response	65
10.2	Plagiarism Check	74